A Comparative Local Fine-Tuning Benchmark Across Candle, Burn, PyTorch, and MLX on Apple Silicon

Hamze Ghalebi

April 2026

Abstract

This paper presents a reproducible local benchmark for supervised fine-tuning across four training backends: Candle, Burn, PyTorch, and MLX. The benchmark addresses a deliberately narrow engineering question: under a fixed dataset split, a fixed held-out evaluation slice, and a fixed small-model training recipe, how do the backends differ in post-fine-tuning quality, training wall time, and inference latency? Each backend saves a persisted base checkpoint, fine-tunes from that checkpoint, reevaluates both the base and fine-tuned checkpoints on the same held-out examples, and emits results in a common JSON schema. On the reported runs, all four backends increase ROUGE-L on the 20-example held-out slice. MLX records the largest absolute ROUGE-L improvement (0.1232), the shortest training wall time (6.29 s), and the lowest fine-tuned latency (22.86 ms). These findings are descriptive rather than inferential: they characterize one controlled benchmark on one Apple Silicon system and should not be generalized beyond this configuration without further replication.

1 Introduction

Comparisons of local fine-tuning workflows are often based on incomplete proxies, such as training loss alone, results from a single backend, or external inference services that do not evaluate the actual checkpoint produced inside a repository. Such practices make it difficult to answer a practical question that matters in local model development: whether a model is measurably better after fine-tuning than before fine-tuning, and how that answer changes across implementations that nominally follow the same recipe.

This study addresses that question with a controlled benchmark that standardizes the dataset split, prompt–target construction, evaluation slice, metrics, and report schema across four backends: Candle, Burn, PyTorch, and MLX. The purpose is operational rather than theoretical. The goal is not to establish a general ranking of machine-learning frameworks, but to measure how four concrete implementations behave under one fixed local setup.

The benchmark was developed in the context of the "Hackathon: Benchmarking Small Language Models in the Real World," organized by AI Paris Thinker. The results reported here, however, are not official hackathon leaderboard results. They do not use the official evaluation platform, the hidden evaluation dataset, or the official leaderboard scoring path mentioned in the original project context. The paper should therefore be read as an independent local benchmark report.

The main contribution is methodological. The benchmark enforces a common contract for data preparation, training, checkpointing, evaluation, and reporting. That design makes pre-/post-fine-tuning comparisons auditable and reproducible across multiple backends.

2 Benchmark design

The benchmark has two layers: a shared data-and-evaluation contract and backend-specific training implementations. The separation is intentional. It holds the task definition fixed while allowing execution details to vary by framework.

Pipeline. Raw chat JSONL → Rust data cleaning → train/validation JSONL → backend-specific training path → persisted base checkpoint and fine-tuned checkpoint → held-out evaluation → per-backend JSON report → consolidated comparison report and SVG visualization.

The cleaned dataset preserves conversational structure through a `messages` array. For each evaluation example, all messages except the final assistant message form the prompt, and the final assistant message serves as the reference target. Each backend is therefore evaluated on the same continuation task: generate the final assistant turn from the same preceding dialogue context.

This shared contract matters because it separates questions of *implementation* from questions of *task definition*. A change in framework should not alter the evaluation target, the report schema, or the checkpoint comparison protocol.

3 Backend implementations

3.1 Candle

The Candle path is implemented in Rust and stores checkpoints in `safetensors` format. Within this study, it functions as a Rust-native reference backend and demonstrates that the benchmark can be executed without a Python training stack.

3.2 Burn

The Burn path is also implemented in Rust and stores checkpoints in `mpk` format. For this benchmark, autoregressive conditioning is implemented with an encoder-style stack equipped with a causal attention mask. This design choice is specific to the local benchmark implementation and should be interpreted as a practical engineering adaptation rather than as a canonical decoder-only architecture.

3.3 PyTorch

The PyTorch path runs in a dedicated Python 3.12 environment managed by `uv`. The implementation uses token embeddings, positional embeddings, pre-norm multi-head self-attention blocks, causal masking, and AdamW, with MPS acceleration on Apple Silicon when available.

3.4 MLX

The MLX path runs in the same pinned Python 3.12 environment. It uses `mlx.nn.Embedding`, `mlx.nn.MultiHeadAttention`, additive causal masks, `nn.value_and_grad`, and AdamW. In this

repository, MLX provides a second Python-based implementation path that is evaluated under the same benchmark contract as the other backends.

4 Experimental configuration

To isolate backend effects as much as possible, the benchmark fixes the dataset slice, nominal model size, and training schedule across all four implementations.

- Training split: `data/train_mistral.jsonl`
- Validation split: `data/valid_mistral.jsonl`
- Training rows: 512
- Held-out evaluation rows: 20
- Training steps: 800
- Maximum sequence length: 128
- Model dimension: 128
- Attention heads: 4
- Layers: 3
- Hidden dimension: 512
- Maximum generated tokens: 48

All four backends use a deliberately simple word-level tokenization scheme. This is a benchmark convenience rather than a claim of production realism. The choice keeps the comparison self-contained and reduces dependence on external assets or backend-specific tokenizer libraries, but it also lowers the absolute quality ceiling relative to modern subword-based production systems.

5 Evaluation protocol and metrics

Every backend follows the same evaluation sequence:

1. Initialize a base model.
2. Save a persisted base checkpoint.
3. Fine-tune that model on the training split.
4. Save a persisted fine-tuned checkpoint.
5. Reload both checkpoints.
6. Evaluate both checkpoints on the same held-out examples.
7. Emit a report with the same JSON schema.

Backend	Base R-L	FT R-L	Delta	Base Lat.	FT Lat.	Train Time
MLX	0.0042	0.1274	0.1232	52.94	22.86	6.29
PyTorch	0.0040	0.1061	0.1021	189.31	69.72	9.23
Burn	0.0021	0.0963	0.0942	130.40	101.20	21.72
Candle	0.0000	0.0882	0.0882	170.60	91.65	14.05

Table 1: Held-out comparison across the four backends.

Note. ROUGE-L values are unitless. Latencies are in milliseconds. Train time is wall-clock time in seconds for the reported run. Exact match is omitted from the table because it is 0.0 for all four backends.

Backend	FT Avg Len	Len Delta	Final Loss	Min Loss
MLX	32.90	-15.10	1.8379	1.5301
PyTorch	48.00	0.00	4.7964	4.5003
Burn	39.15	-8.85	5.4721	4.6112
Candle	25.95	-22.05	6.2383	5.6218

Table 2: Response-length effects and training-loss summaries.

Note. Length values summarize generated output under the benchmark tokenization. *Len Delta* denotes the fine-tuned average length minus the base-model average length. Loss values are included for traceability but should not be over-interpreted as directly comparable quality metrics across implementations.

The shared metrics are exact match, ROUGE-L, response length, and latency. ROUGE-L, which is based on longest-common-subsequence overlap, is treated as the principal automatic quality metric because the task is open-ended assistant continuation rather than strict string reproduction [1]. Exact match remains useful as a stringent diagnostic, but it is too brittle to be the primary criterion in this setting.

Response length and latency are reported because quality alone is not sufficient for engineering evaluation. A backend that improves overlap at the cost of materially slower generation or systematically distorted output length may not be the most useful choice in practice. Loss values are also retained in the auxiliary summary table, but they should be interpreted as optimization diagnostics rather than as backend-independent measures of downstream quality.

6 Results

The numerical results in Tables 1 and 2 are taken from the consolidated comparison report produced by the repository artifacts.

All four backends show higher ROUGE-L after fine-tuning on the held-out slice. The absolute increases are 0.1232 for MLX, 0.1021 for PyTorch, 0.0942 for Burn, and 0.0882 for Candle. Within the reported runs, MLX therefore records the largest ROUGE-L gain.

Runtime measurements show the same backend leading on the efficiency side of this benchmark. MLX has the shortest training wall time (6.29 s) and the lowest fine-tuned latency (22.86 ms). PyTorch ranks second on both ROUGE-L gain and fine-tuned latency. Burn and Candle remain competitive

as Rust-native baselines, but they are slower and achieve smaller ROUGE-L improvements under the present configuration.

Average output length changes after fine-tuning for all four backends. The fine-tuned PyTorch model preserves the original average output length, whereas the other three fine-tuned models generate shorter continuations on average. This matters for interpretation because lexical-overlap metrics can be influenced by response length and truncation behavior.

Exact match remains 0.0 for every run. In practical terms, this means exact string reproduction does not discriminate among the backends on this task. That result is consistent with the use of an open-ended generation objective and a small held-out slice, where exact match is typically much harsher than overlap-based metrics.

7 Discussion

The primary contribution of the benchmark is procedural rather than algorithmic. Its value lies in making pre-/post-fine-tuning comparisons explicit, auditable, and backend-agnostic. The benchmark does not merely report training loss; it evaluates persisted base and fine-tuned checkpoints on the same held-out prompts and stores the outcomes in a common schema.

Three observations follow from the reported runs.

First, the benchmark is sufficiently discriminative to register positive ROUGE-L shifts for all four backends under the same evaluation protocol. This does not prove that the gains would persist under repeated trials or larger datasets, but it does show that the measurement path is capable of capturing post-fine-tuning changes on the current task.

Second, the comparison is inherently multi-objective. No single metric adequately describes backend behavior. Quality gain, training wall time, response length, and latency all matter, and they do not collapse to the same ordering in every benchmark one might design.

Third, the ranking is local to the tested system. The results are best interpreted as a property of this Apple Silicon machine, this implementation set, this tokenization scheme, and this training recipe. They should not be read as framework-wide conclusions beyond the present benchmark.

8 Threats to validity

Several limitations constrain the scope of the conclusions.

- **Small model scale.** The benchmark uses an intentionally small model. The reported values therefore do not estimate the behavior of larger production-scale checkpoints.
- **Simplified tokenization.** The word-level tokenization scheme improves reproducibility and self-containment, but it is not representative of modern production tokenization.
- **Small held-out slice.** The evaluation set contains only 20 examples. The reported ordering should therefore be read as descriptive and directional rather than definitive.
- **Single reported run per backend.** The paper does not estimate variance across random

seeds or repeated trials. No confidence intervals or significance tests are reported.

- **Implementation non-identity.** The backends are behaviorally aligned under a shared contract, but they are not numerically identical implementations of one execution graph.
- **Automatic-metric limits.** ROUGE-L captures lexical overlap rather than full semantic adequacy, factual correctness, or instruction quality. Human evaluation would be required for stronger claims about response quality.

Taken together, these limitations imply a narrow interpretation: the paper reports how four concrete backends behave on one controlled local fine-tuning benchmark, not how they will behave in general.

9 Reproducibility

The benchmark is designed so that the reported comparison can be regenerated from repository commands. The commands used to produce the underlying artifacts are listed below.

```
cargo run --features candle --bin 07_candle_true_pre_post -- \
  --train-path data/train_mistral.jsonl \
  --valid-path data/valid_mistral.jsonl \
  --run-dir artifacts/candle-true-pre-post \
  --train-limit 512 --eval-limit 20 --steps 800 \
  --max-seq-len 128 --d-model 128 --n-layers 3 \
  --mlp-hidden 512 --max-new-tokens 48

cargo run --features burn --bin 08_burn_true_pre_post -- \
  --train-path data/train_mistral.jsonl \
  --valid-path data/valid_mistral.jsonl \
  --run-dir artifacts/burn-true-pre-post \
  --train-limit 512 --eval-limit 20 --steps 800 \
  --max-seq-len 128 --d-model 128 --n-heads 4 \
  --n-layers 3 --mlp-hidden 512 --max-new-tokens 48

.venv-ml/bin/python scripts/true_pre_post_pytorch.py \
  --train-path data/train_mistral.jsonl \
  --valid-path data/valid_mistral.jsonl \
  --run-dir artifacts/pytorch-true-pre-post \
  --train-limit 512 --eval-limit 20 --steps 800 \
  --max-seq-len 128 --d-model 128 --n-heads 4 \
  --n-layers 3 --mlp-hidden 512 --max-new-tokens 48

.venv-ml/bin/python scripts/true_pre_post_mlx.py \
  --train-path data/train_mistral.jsonl \
  --valid-path data/valid_mistral.jsonl \
  --run-dir artifacts/mlx-true-pre-post \
  --train-limit 512 --eval-limit 20 --steps 800 \
  --max-seq-len 128 --d-model 128 --n-heads 4 \
  --n-layers 3 --mlp-hidden 512 --max-new-tokens 48

cargo run --features "candle burn" --bin 10_compare_all_backends -- \
```

```
  --report artifacts/candle-true-pre-post/true_pre_post_report.json \
  --report artifacts/burn-true-pre-post/true_pre_post_report.json \
  --report artifacts/pytorch-true-pre-post/true_pre_post_report.json \
  --report artifacts/mlx-true-pre-post/true_pre_post_report.json \
  --out-dir artifacts/all-backend-comparison
```

The consolidated report used for the paper is stored at `artifacts/all-backend-comparison/all_backend_comparison_report.json`.

10 Conclusion

This paper reports a controlled local fine-tuning benchmark across Candle, Burn, PyTorch, and MLX. The benchmark compares persisted base and fine-tuned checkpoints under a shared evaluation contract and summarizes the results in a backend-agnostic report format.

Within the reported setup, MLX achieves the strongest overall combination of ROUGE-L improvement, training wall time, and fine-tuned latency. PyTorch is the closest alternative in this benchmark, while Candle and Burn remain useful Rust-native baselines for local experimentation and pipeline development.

More broadly, the contribution of the work is the benchmark method itself. By standardizing the dataset split, continuation target construction, metrics, and report schema, the repository becomes an evidence-producing framework for comparing local fine-tuning paths rather than a collection of disconnected training scripts. Future work should expand the evaluation slice, repeat runs across seeds, and add complementary metrics or human assessment before drawing stronger conclusions.

References

[1] Chin-Yew Lin. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, 2004. Association for Computational Linguistics.

[2] Project artifacts. Consolidated backend comparison report: `artifacts/all-backend-comparison/all_backend_comparison_report.json`.

[3] Project architecture note. `ARCHITECTURE.md`.

[4] PyTorch benchmark implementation. `scripts/true_pre_post_pytorch.py`.

[5] MLX benchmark implementation. `scripts/true_pre_post_mlx.py`.